



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Experiment No.: 4 Name: Dhanish Ladwani PRN: 24070521020

1. Experiment Title:

Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required:

Google Chrome (Web Browser), Visual Studio Code (Code Editor), Browser Developer Tools / Console, HTML, CSS Javascript

3. Experiment Program Code:

Experiment 4.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Palindrome Checker</title>
  <link rel="stylesheet" href="Experiment 4.css">
</head>

<body>
  <div class="container">
    <div class="card">
      <h2>Palindrome Checker</h2>
      <form id="palindromeForm" onsubmit="event.preventDefault(); checkPalindrome();">
        <div class="form-group">
          <label for="word">Enter a String</label>
          <input type="text" id="word" placeholder="Enter word or text">
        </div>

        <input type="button" class="btn" value="Check Palindrome" onclick="checkPalindrome()">
      </form>

      <div id="result"></div>
    </div>

    <footer style="text-align: center; margin-top: 30px; font-size: 14px; color: gray; border-top: 1px
solid lightgray; padding-top: 15px;">
      Made by Dhanish Ladwani PRN - 24070521020
    </footer>
  </div>

  <script src="Experiment 4.js"></script>
</body>

</html>
```

Experiment 4.css

```
body {
  background-color: whitesmoke;
  color: black;
  font-family: sans-serif;
```

```
margin: 0;
padding: 0;
display: flex;
justify-content: center;
align-items: center;
min-height: 100vh;
}

.container {
  width: 100%;
  max-width: 400px;
  padding: 20px;
}

.card {
  background-color: white;
  padding: 30px;
  border: 1px solid lightgray;
  border-radius: 12px;
  box-shadow: 0 4px 12px lightgray;
}

h2 {
  color: darkblue;
  text-align: center;
  margin-top: 0;
  margin-bottom: 20px;
}

hr {
  border: 0;
  height: 1px;
  background-color: lightgray;
  margin: 20px 0;
}

form {
  display: flex;
  flex-direction: column;
  gap: 15px;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

label {
  font-weight: bold;
  font-size: 14px;
  color: dimgray;
}

input[type="text"] {
  padding: 10px;
  border: 1px solid lightgray;
  border-radius: 6px;
  font-size: 16px;
}

input[type="text"]:focus {
  border-color: blue;
  outline: none;
}

.btn,
input[type="button"] {
  padding: 12px;
```

```

background-color: royalblue;
color: white;
border: none;
border-radius: 6px;
font-size: 16px;
font-weight: bold;
cursor: pointer;
transition: background-color 0.2s;
width: 100%;
}

.btn:hover,
input[type="button"]:hover {
    background-color: blue;
}

#result {
    display: none;
    margin-top: 20px;
}

.result-details p {
    margin: 6px 0;
    font-size: 16px;
}

.status-pass {
    color: green;
}

.status-fail {
    color: red;
}

```

Experiment 4.js

```

function reverseString(str) {
    return str.split("").reverse().join("");
}

function message() {
    var msg = "Palindrome Checker";
    function display() {
        return msg;
    }
    return display;
}

function checkPalindrome() {
    try {
        var word = document.getElementById("word").value.trim();
        var resultDiv = document.getElementById("result");

        if (word == "") {
            throw "Please enter a string";
        }
        for (let i = 0; i < word.length; i++) {
            let ch = word[i];
            let code = ch.charCodeAt(0);
            let isAlphaNumeric = (code >= 48 && code <= 57) ||
                (code >= 65 && code <= 90) ||
                (code >= 97 && code <= 122);
            if (!isAlphaNumeric) {
                throw "Only alphabets/digits are allowed!";
            }
        }
        if (word.length < 3) {
            throw "Please enter at least 3 characters";
        }
    }
}

```

```
}
if (word.length > 30) {
    throw "Maximum 30 characters allowed";
}

let input = word.toLowerCase();
var reverse = reverseString(input);
var isPalindrome = (input === reverse);

resultDiv.style.display = "block";
resultDiv.className = isPalindrome ? "status-pass" : "status-fail";
resultDiv.innerHTML = `
    <hr>
    <div class="result-details">
        <p><strong>Word:</strong> ${word}</p>
        <p><strong>Reversed:</strong> ${reverse}</p>
        <p><strong>Result:</strong> ${isPalindrome ? "PALINDROME" : "NOT A PALINDROME"}</p>
    </div>
`;
}
catch (error) {
    alert(error);
}
}
```

4. Output:

The image displays three sequential screenshots of a web browser showing a "Palindrome Checker" application. The browser's address bar indicates the file path: `C:/Users/Admin/Downloads/Javascript/Experiment%204/experiment%204/Experiment%204.html`. The application interface includes a title "Palindrome Checker", an input field labeled "Enter a String", a "Check Palindrome" button, and a display area for the word, its reverse, and the final result.

First Screenshot: The input field contains "madam". The output shows:
Word: madam
Reversed: madam
Result: PALINDROME

Second Screenshot: The input field contains "Dhanish". The output shows:
Word: Dhanish
Reversed: hsinahd
Result: NOT A PALINDROME

Third Screenshot: The input field contains "Racecar". The output shows:
Word: Racecar
Reversed: racecar
Result: PALINDROME

Each screenshot also shows a footer text: "Made by Dhanish Ladwani PRN - 24070521020".

5. Result/Conclusion:

Function types, variable scope and closures were demonstrated using a nested function (message/display) that returns an inner function retaining access to its outer variable, illustrating closure behaviour. A palindrome checker was built using a named function to reverse the input string, with try-catch blocks used to handle invalid input such as empty strings, non-alphanumeric characters, and strings outside the allowed length range, raising and catching custom error messages via throw. This experiment reinforced the concepts of function scope, closures and exception handling in JavaScript.